St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2024-2025
Class: BE-ITA/B, Semester: VII
Subject: Data Science Lab

Experiment – 1

1. Aim: To implement a Bayes' Network for Alarm problem

2. Objectives: Students should be able to apply to apply reasoning for a problem in an uncertain domain.

3. Prerequisite: Python basics

4. Requirements: PC, Python 3.9, Windows 10/ MacOS/ Linux, IDLE IDE

5. Pre-Experiment Exercise:

Theory:

Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty. We can define a Bayesian network as:

"A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."It is also called a Bayes network, belief network, decision network, or Bayesian model.

Bayesian networks are probabilistic, because these networks are built from a probability distribution, and also use probability theory for prediction and anomaly detection.

Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network. It can also be used in various tasks including prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction, and decision making under uncertainty.

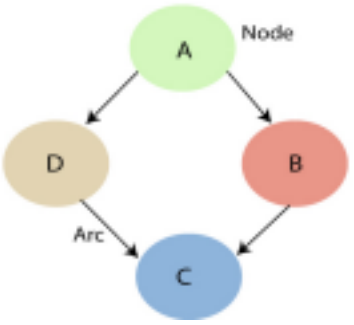
Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

- Directed Acyclic Graph
- Table of conditional probabilities.

The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an Influence diagram.

Bayesian Belief Network:

A Bayesian network graph is made up of nodes and Arcs (directed links),



where:

- Each node corresponds to the random variables, and a variable can be continuous or

discrete.

- Arc or directed arrows represent the causal relationship or conditional probabilities between random variables. These directed links or arrows connect the pair of nodes in the graph.
- These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other
- In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.
- If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.
- **Node C is independent of node A.**

The Bayesian network has mainly two components:

- Causal Component
- Actual numbers.

Each node in the Bayesian network has condition probability distribution P(Xi |Parent(Xi)), which determines the effect of the parent on that node.

6. Laboratory Exercise

A. Procedure

Use google colab for programming.
ii. Import prebuilt model for Bayes' Network.
iii.Display Network structure for the problem given as output.
iv.Display conditional probability table.
v. Add relevant comments in your programs and execute the code. Test it for various cases. c

7. Post-Experiments Exercise:

A. Extended Theory:

a. Design a Bayes' network with conditional probability associated with each node for lung cancer problem.
b. Calculate number of independent variables required for a Bayes' Network?

B. Post Lab Program:

a. Write a Python program to implement Bayes' Network for Lung cancer problem.

C. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

D. References:

[1] https://colab.research.google.com/drive/1ZvtaLqPhzZtx5Vjt1UKiWdRlQttHI-M6?usp=sharing
[2]Stuart Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach", Third Edition, Pearson Education

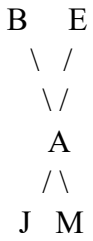
LAB EXERCISE:

A burglar alarm is installed at A's home. This alarm notifies in case of burglary, and can occasionally respond to minor earthquakes. There are 2 neighbours to A, M and J , M and J promise to call A in the office when they hear the alarm. J always calls A when he hears an alarm but many times he is confused with the telephone ring and burglar alarm. M many times misses the alarm because he keeps on listening to loud music given the evidence of who has not or has called. We have to estimate the probability of burglary.

Prior Probabilities

P(B = true) = 0.001 (burglary is rare)

P(E = true) = 0.002 (earthquakes are also rare)



B = Burglary

E = Earthquake

A = Alarm

J = JohnCalls

M = MaryCalls

Burglary/Earthquake	Alarm	
	T	F
T,F	0.95	0.05
T,T	0.94	0.06
F,F	0.29	0.71
F,T	0.0001	0.999

‘

Alarm	J calls	
	T	F
T	0.70	0.30
F	0.01	0.99

Alarm	M calls	
	T	F
T	0.90	0.10
F	0.05	0.95

Python Code for Lab Exercise:

```
!pip install pgmpy
```

```
[17] #importing neccessary modules for bayesian network experiment
     from pgmpy.models.DiscreteBayesianNetwork import DiscreteBayesianNetwork
     from pgmpy.factors.discrete import TabularCPD
     from pgmpy.inference import VariableElimination
```

```
[18] #model_structure
     model = DiscreteBayesianNetwork([
         ('B', 'A'),     # Burglary influences Alarm
         ('E', 'A'),     # Earthquake influences Alarm
         ('A', 'J'),     # Alarm influences JohnCalls
         ('A', 'M')      # Alarm influences MaryCalls
     ])
```

```
[19] # Prior probabilities
     cpd_b = TabularCPD(variable='B', variable_card=2, values=[[0.999], [0.001]])  # B=False, B=True
     cpd_e = TabularCPD(variable='E', variable_card=2, values=[[0.998], [0.002]])  # E=False, E=True
```

```
[20] #conditional probability table for alarm
     cpd_a = TabularCPD(
         variable='A', variable_card=2,
         values=[
             [0.999, 0.71, 0.06, 0.05],  # A=False
             [0.001, 0.29, 0.94, 0.95]   # A=True
         ],
         evidence=['B', 'E'],
         evidence_card=[2, 2]
     )
```

```
#table for john calls
cpd_j = TabularCPD(
    variable='J', variable_card=2,
    values=[
        [0.95, 0.1],  # J=False
        [0.05, 0.9]   # J=True
    ],
    evidence=['A'],
    evidence_card=[2]
)
#table for mary calls
cpd_m = TabularCPD(
    variable='M', variable_card=2,
    values=[
        [0.99, 0.3],  # M=False
        [0.01, 0.7]   # M=True
    ],
    evidence=['A'],
    evidence_card=[2]
)
```

```
[22] #adding table probabilities to model and verifying
     model.add_cpds(cpd_b, cpd_e, cpd_a, cpd_j, cpd_m)
     print("Model valid?", model.check_model())
```

```
Model valid? True
```

0s

```
infer = VariableElimination(model)
posterior = infer.query(['B'], evidence={'J': 1, 'M': 1})
print("\nProbability of Burglary given John and Mary called:")
print(posterior)
```

```
Probability of Burglary given John and Mary called:
+------+----------+
| B    |   phi(B) |
+======+==========+
| B(0) |   0.7158 |
+------+----------+
| B(1) |   0.2842 |
+------+----------+
```

The probability of it being a burglary when both John and Mary call is 28.42%

Post Experiment Exercise

B. Post Lab Program:

a. Write a Python program to implement Bayes' Network for Lung cancer problem.

Question: Consider a situation in which we want to reason about relationship between smoking and lung cancer, we will take 5 boolean random variables, 'C' represents having lung cancer, S represents smoke, RLE represents reduced life expectancy, 'SHS' represents second hand smoke exposure and PS is at least one parent smokes. We know that whether or not a person has cancer is directly influenced by whether she/he is exposed to smoke(S) or second hand smoke(SHS). Both of these things are influenced by whether his or her parents' smoke(PS). Cancer reduces a person's life expectancy.Calculate probability of cancer if atleast parent smokes.

```
[1] !pip install pgmpy
```

```
[2] from pgmpy.models.DiscreteBayesianNetwork import DiscreteBayesianNetwork
    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
```

```
[3] #defining the model
    model = DiscreteBayesianNetwork([
        ('PS', 'S'),
        ('PS', 'SHS'),
        ('S', 'C'),
        ('SHS', 'C'),
        ('C', 'RLE')
    ])
```

```
#prior probability of parent smoke
cpd_ps = TabularCPD(variable='PS', variable_card=2, values=[[0.8], [0.2]])  # PS=False, PS=True
# P(S | PS)
cpd_s = TabularCPD(
    variable='S', variable_card=2,
    values=[[0.9, 0.4], [0.1, 0.6]],  # S=False, S=True
    evidence=['PS'], evidence_card=[2]
)

# P(SHS | PS)
cpd_shs = TabularCPD(
    variable='SHS', variable_card=2,
    values=[[0.85, 0.3], [0.15, 0.7]],  # SHS=False, SHS=True
    evidence=['PS'], evidence_card=[2]
)

# P(C | S, SHS)
cpd_c = TabularCPD(
    variable='C', variable_card=2,
    values=[
        [0.999, 0.9, 0.9, 0.3],  # C=False
        [0.001, 0.1, 0.1, 0.7]   # C=True
    ],
    evidence=['S', 'SHS'],
    evidence_card=[2, 2]
)

# P(RLE | C)
cpd_rle = TabularCPD(
    variable='RLE', variable_card=2,
    values=[[0.1, 0.95], [0.9, 0.05]],  # RLE=False, RLE=True
    evidence=['C'], evidence_card=[2]
)
```

```
print("\nStructure:", model.edges())
for cpd in model.get_cpds():
    print(cpd)
```

```
+-------+-----+
| PS(0) | 0.8 |
+-------+-----+
| PS(1) | 0.2 |
+-------+-----+
```

```
+------+------+------+
| PS   | PS(0) | PS(1) |
+------+------+------+
| S(0) | 0.9  | 0.4  |
+------+------+------+
| S(1) | 0.1  | 0.6  |
+------+------+------+
```

```
+--------+------+------+
| PS     | PS(0) | PS(1) |
+--------+------+------+
| SHS(0) | 0.85 | 0.3  |
+--------+------+------+
| SHS(1) | 0.15 | 0.7  |
+--------+------+------+
```

```
+------+--------+--------+--------+--------+
| S    | S(0)   | S(0)   | S(1)   | S(1)   |
+------+--------+--------+--------+--------+
| SHS  | SHS(0) | SHS(1) | SHS(0) | SHS(1) |
+------+--------+--------+--------+--------+
| C(0) | 0.999  | 0.9    | 0.9    | 0.3    |
+------+--------+--------+--------+--------+
| C(1) | 0.001  | 0.1    | 0.1    | 0.7    |
+------+--------+--------+--------+--------+
```

```
+--------+------+------+
| C      | C(0) | C(1) |
+--------+------+------+
| RLE(0) | 0.1  | 0.95 |
+--------+------+------+
| RLE(1) | 0.9  | 0.05 |
+--------+------+------+
```

```
model.add_cpds(cpd_ps, cpd_s, cpd_shs, cpd_c, cpd_rle)
print("Model valid: ", model.check_model())
```

```
Model valid:  True
```

```
infer = VariableElimination(model)
result = infer.query(['C'], evidence={'PS': 1})
print("\nProbability of Cancer given parent smokes (PS=True):")
print(result)
```

```
Probability of Cancer given parent smokes (PS=True):
+------+----------+
| C    |   phi(C) |
+======+==========+
| C(0) |   0.6599 |
+------+----------+
| C(1) |   0.3401 |
+------+----------+
```

The probability that a person has cancer if his/her parents smoke is 34.01 %